

Parcial III. Simulación. Trabajo Práctico

Escribir un programa python en POO que simule las siguientes problemas durante un periodo de tiempo indicado por el usuario antes de su ejecución. Se tiene que generar salidas en trazas que puedan ser analizados. **Una vez finalizada la simulación, el programa debe recopilar estas métricas y enviarlas mediante una petición a una API de Inteligencia Artificial (como OpenAI, Gemini u Ollama) para que esta analice los datos y retorne una conclusión y recomendación automatizada que se mostrará por pantalla.**

Se debe tener un menú de opciones que permita acceder a cada problema.

Simulación de Eventos Discretos (8 puntos)

Una fábrica de laptops tiene una única estación de ensamblaje principal y un sistema de recepción donde las órdenes de producción llegan aleatoriamente durante el día. El tiempo de ensamblaje en la estación varía según las especificaciones del equipo, pero sigue una distribución exponencial con una media de 5 minutos. La fábrica también mantiene un inventario limitado de componentes esenciales, como procesadores de gama alta, que deben ser reabastecidos cuando el nivel de stock cae por debajo de un umbral crítico.

Se requiere realizar una simulación para evaluar la eficiencia del sistema de ensamblaje y la gestión del inventario, con el objetivo de minimizar los tiempos de espera de las órdenes de producción y evitar detener la línea por falta de componentes. La simulación debe considerar los siguientes factores:

1. **Tiempo de llegada de las órdenes:** Las peticiones de laptops llegan de acuerdo con una tasa promedio de 10 órdenes por hora (distribución Poisson).
2. **Tiempo de servicio de ensamblaje:** El tiempo de ensamblaje sigue una distribución exponencial con una media de 5 minutos por laptop.
3. **Nivel de inventario:** El inventario de procesadores se reabastece automáticamente cada vez que cae por debajo del umbral de 10 unidades. El tiempo de reabastecimiento (entrega del proveedor) es de 15 minutos, y los componentes son entregados en lotes de 50 unidades.
4. **Objetivo:** Determinar el promedio de tiempo de espera de las órdenes, el tiempo total que toma despachar un lote, y el número de veces que la línea se detiene por falta de stock.

La simulación debe ejecutarse durante un período de horas indicado por el usuario y generar métricas clave como el tiempo de espera promedio, la cantidad de órdenes retrasadas por falta de inventario y la eficiencia en el reabastecimiento.

Simulación Continua (8 puntos)

Un centro de datos necesita simular el proceso térmico y de rendimiento de un clúster de servidores de alto rendimiento. El clúster recibe un flujo continuo de tráfico de red (peticiones y cálculos) que genera una carga constante de procesamiento. La tasa de entrada de tráfico y la temperatura del servidor son factores clave que afectan la cantidad de datos procesados (throughput).

El sistema está diseñado para operar a una temperatura constante de 70°C, pero debido a la naturaleza del procesamiento, la temperatura puede fluctuar debido a variaciones en la tasa de tráfico entrante, el calor generado por las CPUs y el sistema de refrigeración líquida. El procesamiento se realiza de manera continua, y se espera que el sistema alcance un equilibrio térmico donde la tasa de generación de calor y la tasa de disipación sean iguales. Sin embargo, el sistema es sensible a cambios en el tráfico, lo que puede generar fluctuaciones de temperatura y activar el estrangulamiento térmico (thermal throttling), afectando severamente el rendimiento.

Se requiere realizar una simulación continua del proceso para evaluar cómo las variaciones en la tasa de tráfico (en función de la demanda) y las fluctuaciones en la temperatura afectan el procesamiento de datos. La simulación debe considerar:

1. **Tasa de entrada de tráfico:** Las peticiones entran al clúster a una tasa variable, entre 1 y 5 Gbps, que depende de la demanda de los usuarios. La tasa de entrada sigue una distribución normal con una media de 3 Gbps y una desviación estándar de 1 Gbps.
2. **Temperatura del servidor:** La temperatura está influenciada por la carga de procesamiento que genera calor y por el sistema de refrigeración que absorbe el calor. La ecuación de balance térmico es una ecuación diferencial de primer orden, que modela cómo la temperatura cambia en el tiempo dependiendo de las entradas de calor y disipación.
3. **Rendimiento del clúster:** La cantidad de datos procesados depende de la temperatura y de la tasa de entrada de tráfico. El sistema se supone que tiene una eficiencia del 90% cuando la temperatura se mantiene estable en 70°C o menos.
4. **Objetivo:** Determinar cómo las fluctuaciones en el tráfico y la temperatura afectan la cantidad de Terabytes procesados y la estabilidad térmica del sistema. Evaluar la viabilidad de operar el clúster de manera continua sin que la temperatura se des controle.

La simulación debe ejecutarse durante un tiempo en horas indicado por el usuario para obtener métricas como la cantidad total de datos procesados, la variabilidad en la temperatura del servidor y la eficiencia global de la red.

Pregunta Bonus (2 puntos adicionales):

Animación de la Simulación en Pygame Desarrollar una interfaz gráfica interactiva utilizando la librería **pygame** que permita visualizar en tiempo real el comportamiento de los dos problemas planteados.

Representación del Sistema: Mostrar gráficamente los elementos clave de la simulación elegida.

- *Si se elige el Problema 1:* Visualizar la cola de órdenes entrantes, el estado de la estación de ensamblaje (ej. cambiando de color si está libre u ocupada) y un indicador visual del nivel de inventario de procesadores.
- *Si se elige el Problema 2:* Visualizar el nodo del servidor y representar las fluctuaciones de temperatura mediante una barra térmica o cambios de color (ej. de azul a rojo crítico) a medida que ingresa el tráfico.

Indicadores en Pantalla (HUD): Renderizar texto dinámico dentro de la ventana de Pygame mostrando las variables críticas actualizadas (tiempo simulado, unidades en inventario, temperatura actual, etc.).

Interactividad: Incluir controles básicos mediante eventos de teclado para pausar o reanudar la animación sin detener el cálculo lógico de la simulación.

Sincronización: La representación visual debe estar conectada directamente a los atributos de las clases desarrolladas en la solución de consola (POO).

Pautas de Evaluación.

1. La evaluación es individual o en equipo de un máximo de tres personas.
2. Usar paradigma de programación orientada a objetos
3. Consumir Api de la IA y generar conclusión y archivos de texto: 4 ptos.
4. Utilizar repositorios github.
5. Los códigos iguales tendrán una penalización de puntos menos.
6. La entrega y defensa se realizará de forma presencial en hora de clases.
7. Realizar validaciones de datos introducidos por el usuario en los comandos y datos cargados.
8. El código deberá estar comentado.
9. Tener datos por defectos para tomarlos como prueba.